

5-4 · DataFrame과 SparkSQL 가공 · 정리·회고 0.5H

정리·회고

세 도구를 한 표로, 5-5 를 위해 파티션 수를 본다

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

30분 뒤에는

- ① Pandas · PySpark API · SQL 문법을 한 표로 나란히 정리한다
- ② 오늘 만든 테이블 세 개와 셔플 파티션 수를 확인한다
- ③ 자주 겪는 오류 네 가지의 원인과 해결을 안다
- ④ 5-5 파티셔닝·셔플링 최적화의 준비를 확인한다

데이터가 5행인데 200조각으로 나누는 것이 얼마나 낭비인지 — 5-5의 출발점

세 도구 대조표 — 이름이 거의 그대로

하려는 일	Pandas	PySpark API	SQL
열 고르기	df[['a']]	df.select("a")	SELECT a
필터	df[df.a>1]	df.filter(F.col("a")>1)	WHERE a>1
파생	df.assign(c=...)	df.withColumn("c", ...)	SELECT ... AS c
집계	groupby('g').agg()	groupBy("g").agg()	GROUP BY g
조인	merge(b, on='k')	join(b, on="k")	JOIN ... ON
윈도	rolling / rank	F.row_number().over(w)	OVER (PARTITION BY ...)
정렬	sort_values('a')	orderBy("a")	ORDER BY a
저장	to_parquet()	writeTo(...).createOrReplace()	CREATE TABLE ... AS

헛갈리면 「Pandas 로는 뭐였지」를 먼저 떠올리세요 — 다른 점은 자연 평가와 F.col 정도

다음 차시를 위한 준비 — 테이블과 파티션 수

1 **spark.sql("SHOW TABLES IN lake.db").show()**
orders · product_sales · daily_revenue 세 개가 보인다

2 **sales.rdd.getNumPartitions()**
sales 의 파티션 수 — 5-5 의 주제

3 **spark.conf.get("spark.sql.shuffle.partitions")**
200 으로 나온다 — 데이터 5행을 200조각으로 나누고 있다

셔플 파티션

200

여덟 개를 스스로 확인해 보세요

- 1 PySpark 문법이 Pandas 와 거의 같은 이름이라는 것을 확인했다
- 2 API 와 SQL 이 같은 논리 계획으로 수렴한다는 것을 이해했다
- 3 `join` · `withColumn` · `filter` · `groupBy` 를 체인으로 이어 파이프라인을 만들었다
- 4 조인 키를 문자열로 지정해 중복 컬럼 없이 조인했다
- 5 조인 전후 행 수와 오른쪽 키의 고유성을 직접 확인했다
- 6 윈도우 함수로 날짜별 순위를 매겨 봤다
- 7 같은 집계를 API 와 SQL 로 작성하고 `explain()` 구조가 같음을 확인했다
- 8 CTE 로 나눈 SQL 을 쓰고 가공 결과를 Iceberg 테이블로 저장했다

증상으로 찾는 원인과 해결

증상	원인	해결
cannot resolve 컬럼명	오타이거나 가공 전 DataFrame 을 참조	직전 DataFrame 에 printSchema()
Reference 'product_id' is ambiguous	양쪽에 같은 이름의 컬럼이 남았다	on="product_id" 문자열 또는 alias("o")·alias("p")
결과가 없거나 ...으로 잘림	action 미호출 · show() 기본 20행	show() 붙이기 · show(truncate=False)
조인 후 행 수가 원본보다 많다	오른쪽 키에 중복	count 와 distinct().count() 비교 · 4-3 방식으로 정리

네 가지 모두 「그 시점의 DataFrame 에 무엇이 있나」를 printSchema() · count() 로 먼저 보면 잡힌다

자주 나오는 질문 네 가지

1 API 와 SQL 중 뭘 배워야 하나요?

둘 다 — SQL 은 2주차 MySQL · 3주차 DuckDB · 4주차 Trino 에서도 쓰이니 투자 대비 효과가 크다

2 임시 뷰와 Iceberg 테이블은 뭐가 다른가요?

임시 뷰는 세션이 끝나면 사라진다 — Iceberg 테이블은 MinIO 에 파일로 남는다

3 F.col 없이 문자열로 써도 되나요?

간단한 조건은 된다 — 이름에 공백이 있거나 복잡한 식에서는 F.col 이 안전

4 Pandas 코드를 그대로 옮길 수 있나요?

거의 — 인덱스가 없고, apply 같은 행 단위 함수는 느리니 내장 함수로

더 해 볼 것 넷 — 5-5 연습

01 날짜별 매출 두 방식

API · SQL
결과 대조

02 Spark UI 그래프

SQL 탭에서
Exchange 찾기

03 products 를 크게

Broadcast →
SortMerge 지점

04 5-3 의 orders

lake.db.orders 를
오늘 방식으로

수만 행으로 키우면 조인 방식이 BroadcastHashJoin 에서 SortMergeJoin 으로 바뀌는 지점이 있다

다음 회차 예고

5-5 · 파티셔닝과 셔플링 최적화

01

이론 1H

파티셔닝과
셔플링 원리

02

실습 1.5H

repartition·
coalesce 셔플

03

실습 1H

배치 스크립트로
데이터 마트 산출

04

정리 0.5H

정리·회고
산출물 점검

준비: 5-3~5-4 compose(스파크 클러스터 + MinIO) · lake.db.orders · product_sales — 배치 스크립트와 마트 테이블이 주차 산출물

5-4 · 정리·회고 0.5H 마무리

다음 회차는 5-5 파티셔닝과 셔플링

「왜 느린가」를 파티션과 셔플로 다루고, 지금까지의 가공을 배치 스크립트로 묶어 5주차 산출물 데이터 마트를 만듭니다

✓ 파티션 = 태스크 병렬성의 단위

✓ 좁은 변환·넓은 변환과 셔플 지점

✓ repartition 과 coalesce 의 차이

✓ Spark UI 의 Shuffle Read/Write

준비할 것

compose 기동 · orders·product_sales